Multi – level Linked List

–Memory Reference and Freeing Memory

```cpp
#include <cstdlib> // Required for malloc, free, exit
#include <iostream>

using namespace std;

// 1. Define the Multilevel Node
struct Node
{
    int data;
    struct Node *next;  // Points to the right (Same Level)
    struct Node *child; // Points down (Sub Level)
};
```

***What the above code depicts*: –**

→ ***struct whose name is Node*:**

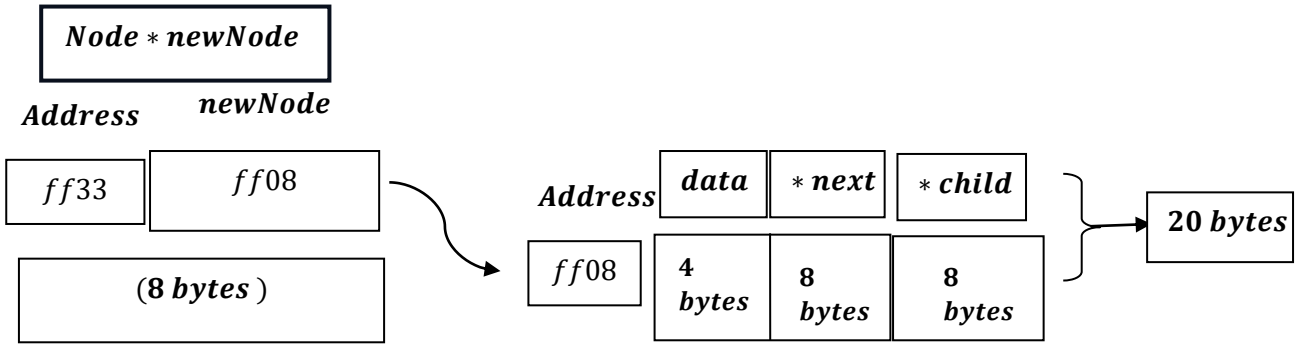
→ ***for struct Node , there will be a memory allocation for integer variable `data`.***

→ ***for struct Node, there will be a memory allocation for pointer next.***

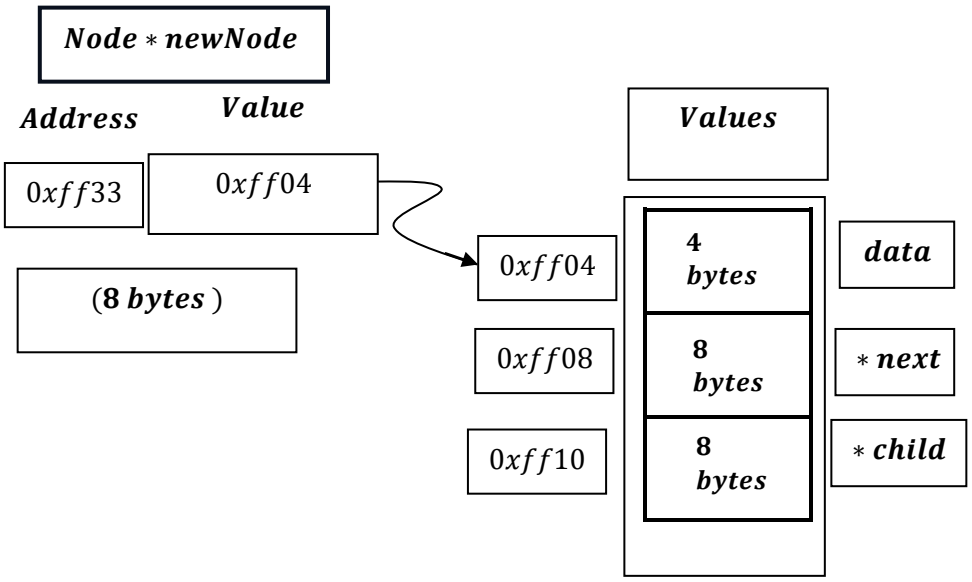
→ ***for struct Node, there will be a memory allocation for pointer child.***

```cpp
Node *createNode(int data)
{
    Node *newNode = (Node *)malloc(sizeof(Node));
    if (newNode == nullptr)
    {
        cout << "Memory allocation failed." << endl;
        exit(1);
    }
    newNode->data = data;
    newNode->next = nullptr;
    newNode->child = nullptr;
    return newNode;
}
```

***We know*[*Based on 64 bit system*]:**



***But memory allocation doesn't take like this, actually, this is for understanding, and for explanation, memory allocation a little bit more complex*:**



At now we see newNode pointer , only pointing $0xff04$, but it should update Data part and Next pointer part, how that happens?

→ The compiler knows that the data member is at the very beginning of the struct. Its offset from the start is 0 bytes.

such as: $newNode \rightarrow data = 10$.

$$\rightarrow baseaddress + offset = 0xff04 + 0 = 0xff04.$$

<u>The Compiler's "Secret" Calculation</u>

When the compiler sees newNode->data =10;, it thinks:

- *"Get the base memory address stored in the pointer newNode." (e.g., 0xff04)*
- *"Look up the definition of the Node struct. The data member is at the beginning, so its offset is 0."*
- *"Calculate the final address: base address + offset => 0xff04 + 0 => 0xff04."*
- *"Write the value `10` to the memory location starting at 0xff04."*

Next is next, a pointer variable :

This comes after the integer named data variable $(4\ bytes)$. ***Its offset is*** $4\ bytes$.

$$say, newNode \rightarrow next = nullptr;$$

$$\rightarrow baseaddress + offset = 0xff04 + 4\ bytes = 0xff08.$$

$$\rightarrow And\ then\ write\ head\ pointer's\ value\ to\ address\ 0xff08.$$

When it sees newNode->next = address_value;, it thinks:

1. *"Get the base memory address stored in the pointer newNode." (e.g., 0xff04)*
2. *"Look up the definition of the Node struct. The next member comes after integer variable, which is 4 bytes long. So, the offset for next is 4."*
3. *"Calculate the final address: base address + offset => 0xff04 + 4 => 0xff08."*
4. *"Write the address_value to the memory location starting at 0xff08."*

But this doesn't means , value of newNode pointer gets changed and it starts to point to address of next pointer variable, it continue to point to integer `data` variable .

$$\textbf{\textit{Next is child, a pointer variable}}:$$

$$\textbf{\textit{This comes after next}}\ (\mathbf{8}\ \textbf{\textit{bytes}})\ \textbf{\textit{and data}}\ (\mathbf{4}\ \textbf{\textit{bytes}}).\ \textbf{\textit{Its offset is}}\ \mathbf{12}\ \textbf{\textit{bytes}}\ (\mathbf{8}\ +\ \mathbf{4}).$$

$$\textbf{\textit{say}}, newNode \rightarrow child = nullptr;$$

$$\rightarrow baseaddress + offset = \mathbf{0}xff\mathbf{04} + \mathbf{12}\ \textbf{\textit{bytes}}$$

$$= \mathbf{0}xff\mathbf{04} + \mathbf{0}x\mathbf{0}C = \mathbf{0}xff\mathbf{10}\begin{bmatrix}\mathbf{08} + \mathbf{04} = \mathbf{12} \\ \textbf{\textit{And integer}}\ \mathbf{12}\ \approx \textbf{\textit{hex}}\ \mathbf{0}x\mathbf{0}C\end{bmatrix}.$$

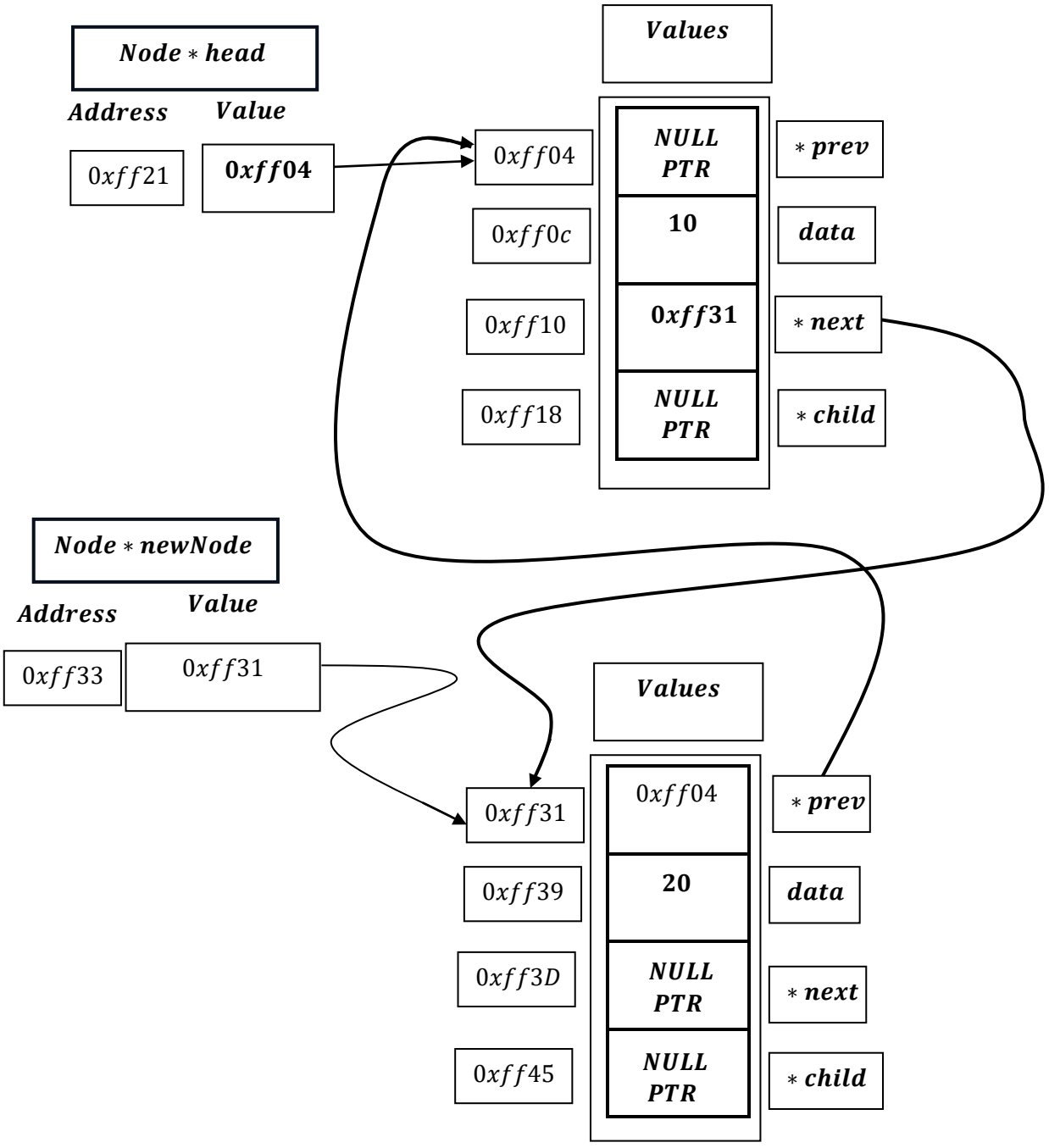
$$\rightarrow \textbf{\textit{And then write head pointer's value to address}}\ \mathbf{0}xff\mathbf{10}.$$

When it sees newNode->next = nullptr;, it thinks:

1. *"Get the base memory address stored in the pointer newNode." (e.g., 0xff04)*
2. *"Look up the definition of the Node struct. The next `child` member comes after next pointer and integer data, which is (8+4) bytes long. So, the offset for next is (8+4)=12."*
3. *"Calculate the final address: base address + offset => 0xff04 + 12 => 0xff10."*
4. *" Write the value `nullptr` or NULL basically (whose output is 0)[pointer points to nothing] to the memory location starting at 0xff10."*

But this doesn't means , value of newNode pointer gets changed and it starts to point to address of child pointer variable, it continue to point to integer data .

Now, accessing from integer data value from newNode pointer:



1. $newNode \rightarrow prev \rightarrow prev$; **Now there is two path**:

$\rightarrow newNode \rightarrow prev: 0xff31 + 0\ offset = 0xff31 \rightarrow 0xff04 \begin{bmatrix} value \\ stored \\ in \\ memory \end{bmatrix}$

$and\ newNode \rightarrow prev(0xff31 + 0\ offset) \rightarrow prev \coloneqq (0xff04) \rightarrow prev: 0xff04 + 0$

$= 0xff04 = nullptr \begin{bmatrix} value \\ stored \\ in \\ memory \end{bmatrix}.$

$\therefore$ *It returns NULLPTR through* $newNode \rightarrow prev \rightarrow prev$.

2. $newNode \rightarrow prev \rightarrow data$; **Now there is two path**:

$\rightarrow newNode \rightarrow prev: 0xff31 + 0\ offset = 0xff31 \rightarrow 0xff04 \begin{bmatrix} value \\ stored \\ in \\ memory \end{bmatrix}$

$and\ newNode \rightarrow prev(0xff31 + 0\ offset) \rightarrow data \coloneqq (0xff04) \rightarrow data: 0xff04 + (0 + 8)\ bytes$

$= 0xff0c = 10 \begin{bmatrix} value \\ stored \\ in \\ memory \end{bmatrix}.$

$\therefore$ *It returns* 10 *through* $newNode \rightarrow prev \rightarrow data$.

3. $newNode \rightarrow prev \rightarrow next$; **Now there is two path**:

$\rightarrow newNode \rightarrow prev: 0xff31 + 0\ offset = 0xff31 \rightarrow 0xff04 \begin{bmatrix} value \\ stored \\ in \\ memory \end{bmatrix}$

$and\ newNode \rightarrow prev(0xff31 + 0\ offset) \rightarrow next \coloneqq$

$(0xff04) \rightarrow next: 0xff04 + (0 + 8 + 4) bytes\ = 0xff04 + 12\ bytes = 0xff10 =$

$0xff31 \begin{bmatrix} value \\ stored \\ in \\ memory \end{bmatrix}.$

$\therefore$ *It returns* $0xff31$ *through* $newNode \rightarrow prev \rightarrow next$.

$$\mathbf{4}.\boldsymbol{newNode} \rightarrow \boldsymbol{prev} \rightarrow \boldsymbol{child}; \boldsymbol{Now\ there\ is\ two\ path}:$$

$$\rightarrow \boldsymbol{newNode} \rightarrow \boldsymbol{prev}: \mathbf{0}\boldsymbol{xff31} + \mathbf{0}\ \boldsymbol{offset} = \mathbf{0}\boldsymbol{xff31} \rightarrow \mathbf{0}\boldsymbol{xff04} \begin{bmatrix} \boldsymbol{value} \\ \boldsymbol{stored} \\ \boldsymbol{in} \\ \boldsymbol{memory} \end{bmatrix}$$

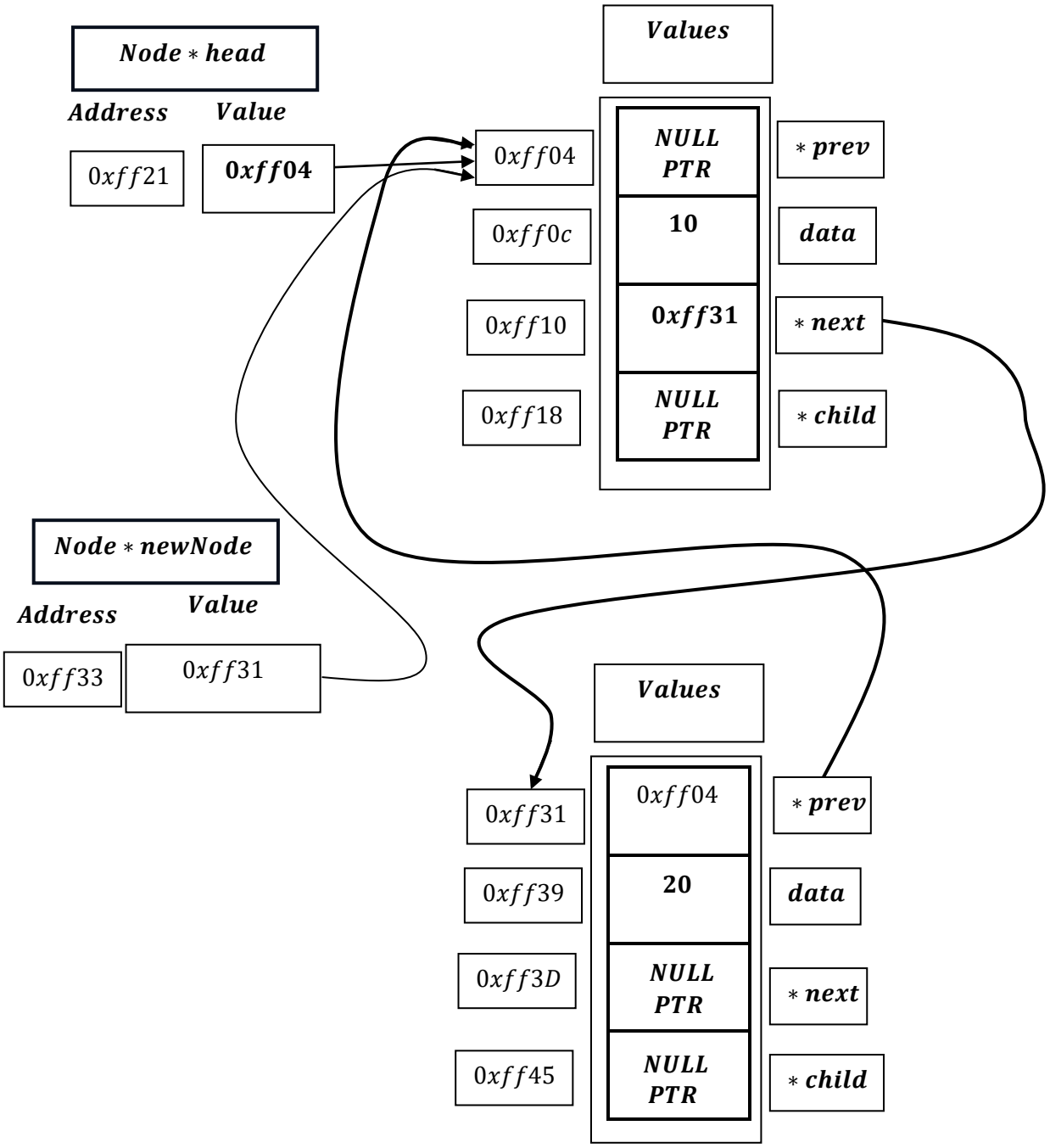
$$\boldsymbol{and\ newNode} \rightarrow \boldsymbol{prev}(\mathbf{0}\boldsymbol{xff31} + \mathbf{0}\ \boldsymbol{offset}) \rightarrow \boldsymbol{child} \coloneqq$$

$$(\mathbf{0}\boldsymbol{xff04}) \rightarrow \boldsymbol{child}: \mathbf{0}\boldsymbol{xff04} + (\mathbf{0} + \mathbf{8} + \mathbf{4} + \mathbf{8})\boldsymbol{bytes} = \mathbf{0}\boldsymbol{xff04} + \mathbf{20}\ \boldsymbol{bytes} = \mathbf{0}\boldsymbol{xff18} =$$

$$\boldsymbol{nullptr} \begin{bmatrix} \boldsymbol{value} \\ \boldsymbol{stored} \\ \boldsymbol{in} \\ \boldsymbol{memory} \end{bmatrix}.$$

$$\therefore \boldsymbol{It\ returns\ `nullptr`\ through}\ \boldsymbol{newNode} \rightarrow \boldsymbol{prev} \rightarrow \boldsymbol{child}.$$

Again,


Node * head
Address
Value
0xff21
0xff04
Values
0xff04
NULL PTR
* prev
0xff0c
10
data
0xff10
0xff31
* next
0xff18
NULL PTR
* child
Node * newNode
Address
Value
0xff33
0xff31
Values
0xff31
0xff04
* prev
0xff39
20
data
0xff3D
NULL PTR
* next
0xff45
NULL PTR
* child

1. $newNode \rightarrow next \rightarrow prev$; **Now there is two path**:

$\rightarrow newNode \rightarrow next: 0xff04 + (0 + 8 + 4)bytes = 0xff04 + 12\ bytes = 0xff10$

$$= 0xff31 \begin{bmatrix} value \\ stored \\ in \\ memory \end{bmatrix}$$

$and\ newNode \rightarrow next[(0xff04 + 12)bytes] \rightarrow prev \coloneqq (0xff10 \coloneqq 0xff31) \rightarrow prev: 0xff31 + 0$

$$= 0xff31 = 0xff04 \begin{bmatrix} value \\ stored \\ in \\ memory \end{bmatrix}.$$

$\therefore It\ returns\ 0xff04\ through\ newNode \rightarrow next \rightarrow prev.$

2. $newNode \rightarrow next \rightarrow data$; **Now there is two path**:

$\rightarrow newNode \rightarrow next: 0xff04 + (0 + 8 + 4)bytes = 0xff04 + 12\ bytes = 0xff10$

$$= 0xff31 \begin{bmatrix} value \\ stored \\ in \\ memory \end{bmatrix}$$

$and\ newNode \rightarrow next[(0xff04 + 12)bytes] \rightarrow data \coloneqq (0xff10 \coloneqq 0xff31) \rightarrow data:$

$$= (0xff31 + (0 + 8)bytes) = 0xff39 = 20 \begin{bmatrix} value \\ stored \\ in \\ memory \end{bmatrix}.$$

$\therefore It\ returns\ 20\ through\ newNode \rightarrow next \rightarrow data.$

3. $newNode \rightarrow next \rightarrow next$; **Now there is two path**:

$\rightarrow newNode \rightarrow next: 0xff04 + (0 + 8 + 4)bytes = 0xff04 + 12\ bytes = 0xff10$

$$= 0xff31 \begin{bmatrix} value \\ stored \\ in \\ memory \end{bmatrix}$$

$and\ newNode \rightarrow next[(0xff04 + 12)bytes] \rightarrow next \coloneqq (0xff10 \coloneqq 0xff31) \rightarrow next:$

$$= (0xff31 + (0 + 8 + 4)bytes) = 0xff3D = NULLPTR \begin{bmatrix} value \\ stored \\ in \\ memory \end{bmatrix}.$$

$\therefore It\ returns\ NULLPTR\ through\ newNode \rightarrow next \rightarrow next.$

4. $newNode \rightarrow next \rightarrow child$; *Now there is two path*:

$\rightarrow newNode \rightarrow next$: $0xff04 + (0 + 8 + 4)bytes = 0xff04 + 12\ bytes = 0xff10$

$$= 0xff31 \begin{bmatrix} value \\ stored \\ in \\ memory \end{bmatrix}$$

and $newNode \rightarrow next[(0xff04 + 12)bytes] \rightarrow child \coloneqq (0xff10 \coloneqq 0xff31) \rightarrow child$:

$$= (0xff31 + (0 + 8 + 4 + 8)bytes) = 0xff45 = NULLPTR \begin{bmatrix} value \\ stored \\ in \\ memory \end{bmatrix}.$$

$\therefore$ *It returns NULLPTR through* $newNode \rightarrow next \rightarrow child$.

How does Heap memory looks like : –

... some memory ...

Address: 0x5000	*integer data value*	*value:* 10
Address: 0x5004	*next pointer value*	*value:* 0x8150
Address: 0x500C	*child pointer value*	*value: NULL*

Offsets: data = 0 (4 bytes), next = 4 (8 bytes), child = 12 (8 bytes) → Total size = 20 bytes

... more scattered memory ...

Address: 0x6240	*integer data value*	*value:* 30
Address: 0x6244	*next pointer value*	*value: NULL*
Address: 0x624C	*child pointer value*	*value: NULL*

Offsets: data = 0 (4 bytes), next = 4 (8 bytes), child = 12 (8 bytes) → Total size = 20 bytes

... even more scattered memory ...

Address: 0x8150	*integer data value*	*value:* 20
Address: 0x8154	*next pointer value*	*value:* 0x6240
Address: 0x815C	*child pointer value*	*value: NULL*

Offsets: data = 0 (4 bytes), next = 4 (8 bytes), child = 12 (8 bytes) → Total size = 20 bytes

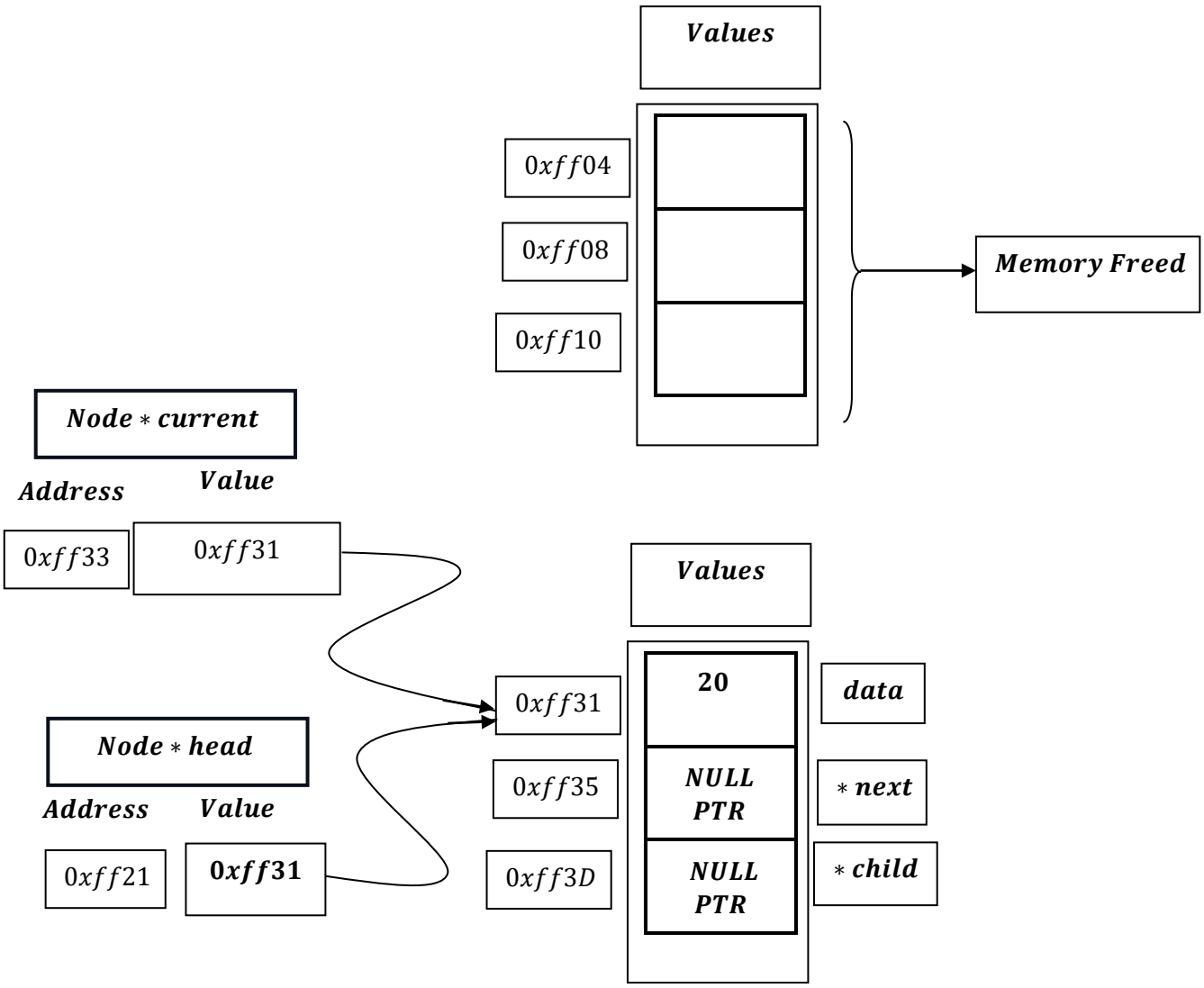
Note: Pointer size = 8 bytes, Integer size = 4 bytes

Freeing up the Memory

```
        free(*current);
```

The free(current) command frees the entire memory block allocated for that node, which includes the space for both next pointer, integer data value and the child pointer.

free () does not know or care that the memory block is being used for a struct with different members. It only knows the starting address (current) and the size of the block that was originally allocated by malloc. It returns that entire chunk of memory to the operating system's heap for reuse.



A simple demonstration of deletion from first node.

***How heap memory looks like*?**